

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 3

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Import the required libraries such as TensorFlow, Keras, Pandas, NumPy, Matplotlib, and Scikit-learn.
2. Load the Heart Disease dataset into a Pandas DataFrame.
3. Explore the dataset and check for missing values and data types.
4. Separate the input features (X) and target variable (y).
5. Standardize the input features using StandardScaler.
6. Split the dataset into training and testing sets using train_test_split().
7. Define a neural network model using the Keras Sequential API with:
8. Input layer
9. Hidden layers with ReLU activation.
10. Output layer with Sigmoid activation
11. Compile the model using the Adam optimizer, Binary Cross-Entropy loss function, and Accuracy as the evaluation metric.
12. Train the model using forward propagation and backpropagation for different learning rates (0.1, 0.01, 0.001, 0.0001) and record the test accuracy.
13. Train the model again using different numbers of epochs (5, 10, 20, 50) while keeping the learning rate constant, and record the performance.
14. Plot graphs showing:

15. Learning Rate vs Accuracy

16. Epochs vs Accuracy

17. Training & Validation Accuracy

18. Training & Validation Loss

19. Evaluate the final model using the test dataset.

20. Generate the Confusion Matrix and Classification Report.

21. Analyze the effect of different learning rates and epochs on the model's performance and draw conclusions.

Code :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

X = df.drop("target", axis=1)
y = df["target"]

scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
```

```

    random_state=42
)

def create_model(lr):
    model = Sequential()
    model.add(Dense(32,activation='relu',input_shape=(13,)))
    model.add(Dense(16,activation='relu'))
    model.add(Dense(8,activation='relu'))
    model.add(Dense(1,activation='sigmoid'))
    optimizer = Adam( learning_rate=lr)
    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

learning_rates = [0.1,0.01,0.001,0.0001]

results=[]
for lr in learning_rates:
    model=create_model(lr)
    history=model.fit(X_train,y_train,epochs=20,
validation_split=0.2,verbose=0
    )

    loss,acc=model.evaluate(X_test,y_test,verbose=0)
    results.append([lr,acc])

learning_rates = [0.1, 0.01, 0.001, 0.0001]

results = []
for lr in learning_rates:
    model = create_model(lr)
    history =
model.fit(X_train,y_train,epochs=20,validation_split=0.2,verbose=0)

    loss, acc = model.evaluate(X_test,y_test,verbose=0)
    results.append([lr, acc])
results = pd.DataFrame(results,
    columns=["Learning Rate", "Accuracy"])

```

```

print(results)

plt.figure(figsize=(7,5))
plt.plot(
    results["Learning Rate"],
    results["Accuracy"],
    marker='o'
)
plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Accuracy")
plt.grid()
plt.show()

epochs = [5, 10, 20, 50]
epoch_results = []

for ep in epochs:
    model = create_model(0.001)
    history = model.fit( X_train,
y_train,epochs=ep,validation_split=0.2,verbose=0)
    loss, acc = model.evaluate(X_test,y_test,verbose=0)

    epoch_results.append([ep, acc])
epoch_results = pd.DataFrame(epoch_results,
    columns=["Epochs", "Accuracy"]
)

print(epoch_results)

plt.figure(figsize=(7,5))
plt.plot(
    epoch_results["Epochs"],
    epoch_results["Accuracy"],
    marker='o'
)

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Accuracy")
plt.grid()

```

```
plt.show()

#training accuracy
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(["Training", "Validation"])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()

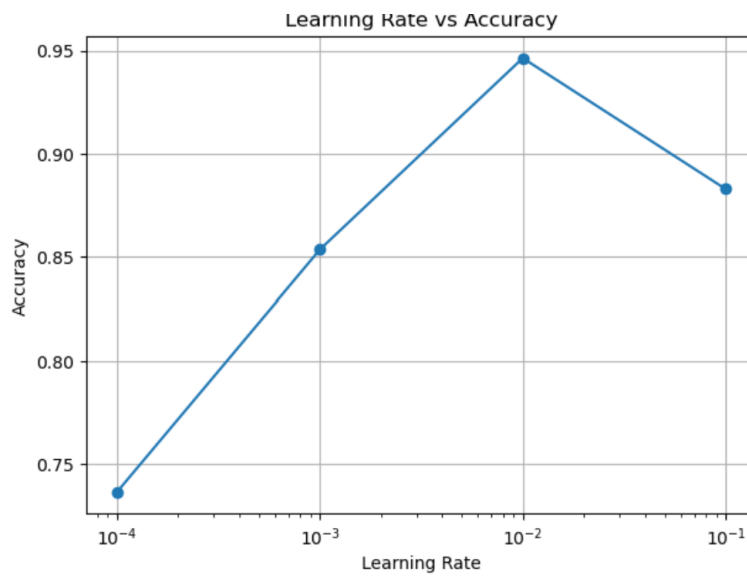
#training loss
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.legend(["Training", "Validation"])
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```

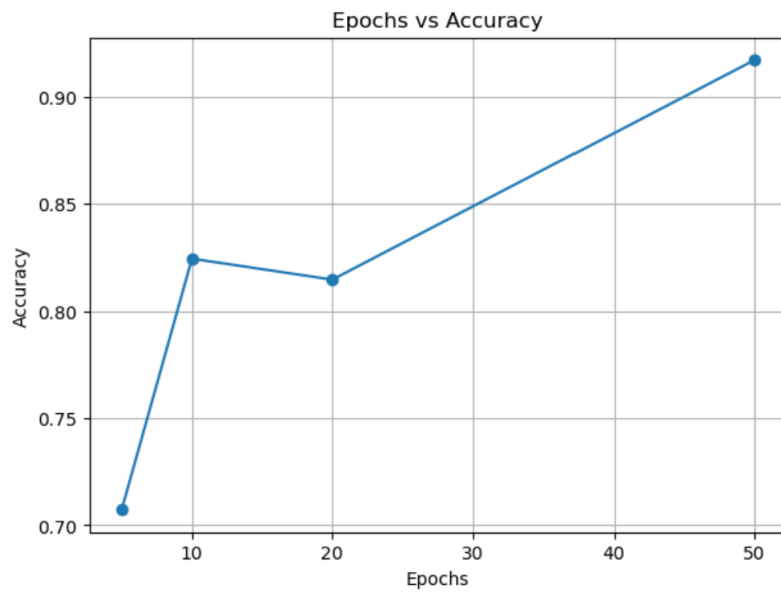
Output :

	Learning Rate	Accuracy
0	0.1000	0.839024
1	0.0100	0.912195
2	0.0010	0.863415
3	0.0001	0.707317

- **Learning rate vs Accuracy visualization:**



- **Epoch vs Accuracy**



- **Training Accuracy Visualization**



- **Training Loss visualization**

